Deliverable #2 : High-Level Architectural Design

SE 3A04: Software Design III – Large System Design

Tutorial Number: T02
Group Number: G5
Group Members:

- Anura, Mithu
- Banik, Tridib
- Khorramijam, Arsia
- Pham, Aaron
- Zhang, Han

1 Introduction

This section should provide an brief overview of the entire document.

1.1 Purpose

The purpose of this document is to present the architectural design and analysis model for the SCEMAS system. It outlines the system's structure, including its subsystems, architectural styles, and class responsibilities.

The document is intended for internal stakeholders, such as project managers, developers, and system designers, involved in the development and implementation of the SCEMAS platform. The prior deliverable, including the Software Requirements Specification (SRS), should be read beforehand to provide additional context for the system requirements.

1.2 System Description

SCEMAS (Smart City Environmental Monitoring and Alert System) is designed to collect, process, and distribute environmental sensor data across a city. The system ingests telemetry data from IoT sensors, validates and stores the data, and makes it available to operators and the public through dashboards and APIs. The platform also includes an alert management system that allows operators to define environmental thresholds and receive notifications when abnormal conditions occur, along with access control mechanisms for managing user accounts, permissions, and device authentication.

The system follows a Presentation-Abstraction-Control (PAC) style architecture, with several subsystems using additional architectural patterns suited to their specific responsibilities. The telemetry ingestion subsystem follows a pipe-and-filter architecture to process sensor data through a sequence of validation and transformation stages. The access management subsystem follows a repository-style architecture, providing a centralized passive data store for managing accounts, permissions, and authentication data. The alert management subsystem follows a blackboard architecture, enabling different components to react to alert conditions and update alert states collaboratively.

1.3 Overview

The remainder of this document describes the architectural structure and design of the SCEMAS system. Section 2 presents the Analysis Class Diagram, illustrating the key classes and their relationships within the system. Section 3 discusses the architectural design of the system, including the overall architecture, subsystem structure, and the reasoning behind key design decisions. Finally, Section 4 presents the Class Responsibility Collaboration (CRC) cards, which further describe the responsibilities of each class and their interactions within the system.

2 Analysis Class Diagram

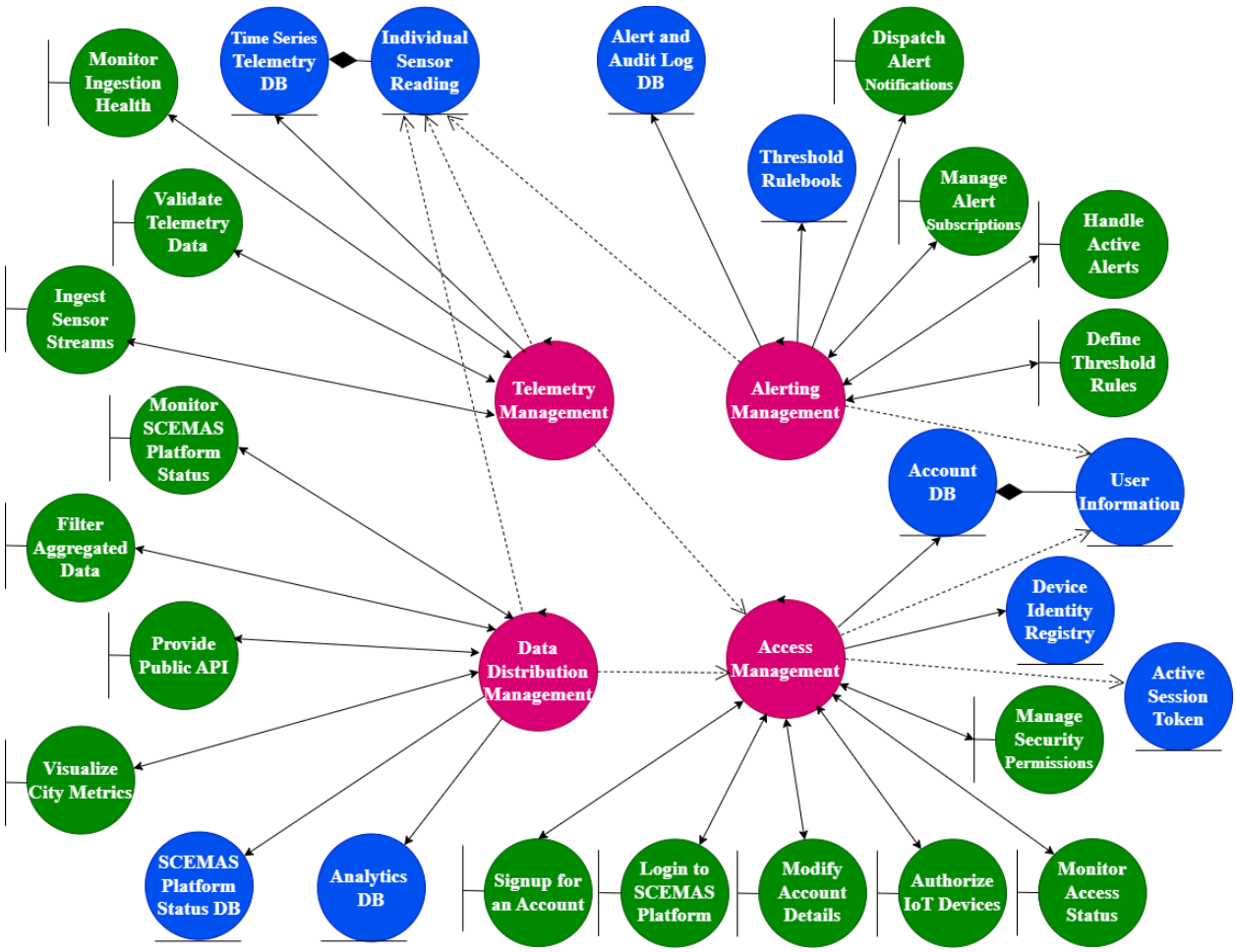


Figure 1: Analysis Class Diagram

3 Architectural Design

3.1 System Architecture

SCEMAS follows an Interaction-Oriented Software Architecture, more specifically best described using the PAC architecture, where the system is decomposed into a set of co-operating agents. The presentation represents what the user sees and interacts with, being the dashboard views. Abstraction holds the key data and domain logic supported by each agent. Control coordinates the logic between each presentation and abstraction, as well as mediating communication between higher and lower-level agents to compose the system hierarchically.

Within PAC architecture, it can be broken down into various agents with differing architectural styles.

Subsystem	**Purpose**	**Architectural Style**
Telemetry Management	Accept incoming data flow and notify required agents	Pipe and Filter
Access Management	Identity verification, sign-up/login, RBAC authentication	Repository
Alerting Management	Define rules, acknowledge results, manage alerts as they originate from exceeding thresholds within the data	Blackboard

PAC was selected as the global architecture for SCEMAS due to a variety of substantial reasons. SCEMAS serves three fundamentally different user roles; City Operator, Public Users, and System Administrators, each requiring a different view of a specific subset of the underlying data. The overall composition supports various simultaneous agents mapping different views with their own independent Abstraction to encourage decision making. For the operator's dashboard, key alert decisions are based on the data being represented in a cohesive way specific to them, where they should not be concerned with other views that represent unrelated data. Unlike a shared-model architecture like MVC that forces all three views to share a single domain model, PAC naturally isolates these interaction models while still allowing coordination through control components ensuring loose coupling.

SCEMAS is inherently data driven, including telemetry updates, rule changes, and alert triggers. Although broken into subsystems that represent different architectures, PAC will support multiple processes of interaction via a root-level Data Distribution Management Controller. Telemetry Management and Alerting Management get data from a shared entity called Individual Sensor Reading. This data from the shared entity is used by the Alerting Management Controller for decision support of whether to trigger any alert or not. Finally, the Data Distribution Management Controller distributes or presents views to the dashboard, which surfaces alerts to operators who acknowledge them. Each agent operates independently, communicating through shared entities rather than direct control-to-control communication. This cleanly separates concerns between agents, ensuring each is only responsible for its own tasks without needing to understand the implementation of other agents. Modularity is also another important feature to SCEMAS to support various dashboard analytics, and/or alert management. PAC ensures new agents can be added to the tree without affecting the existing agents supporting the Open-Closed Principle where additional features or elements will not overwrite existing code but supplement the overall system.

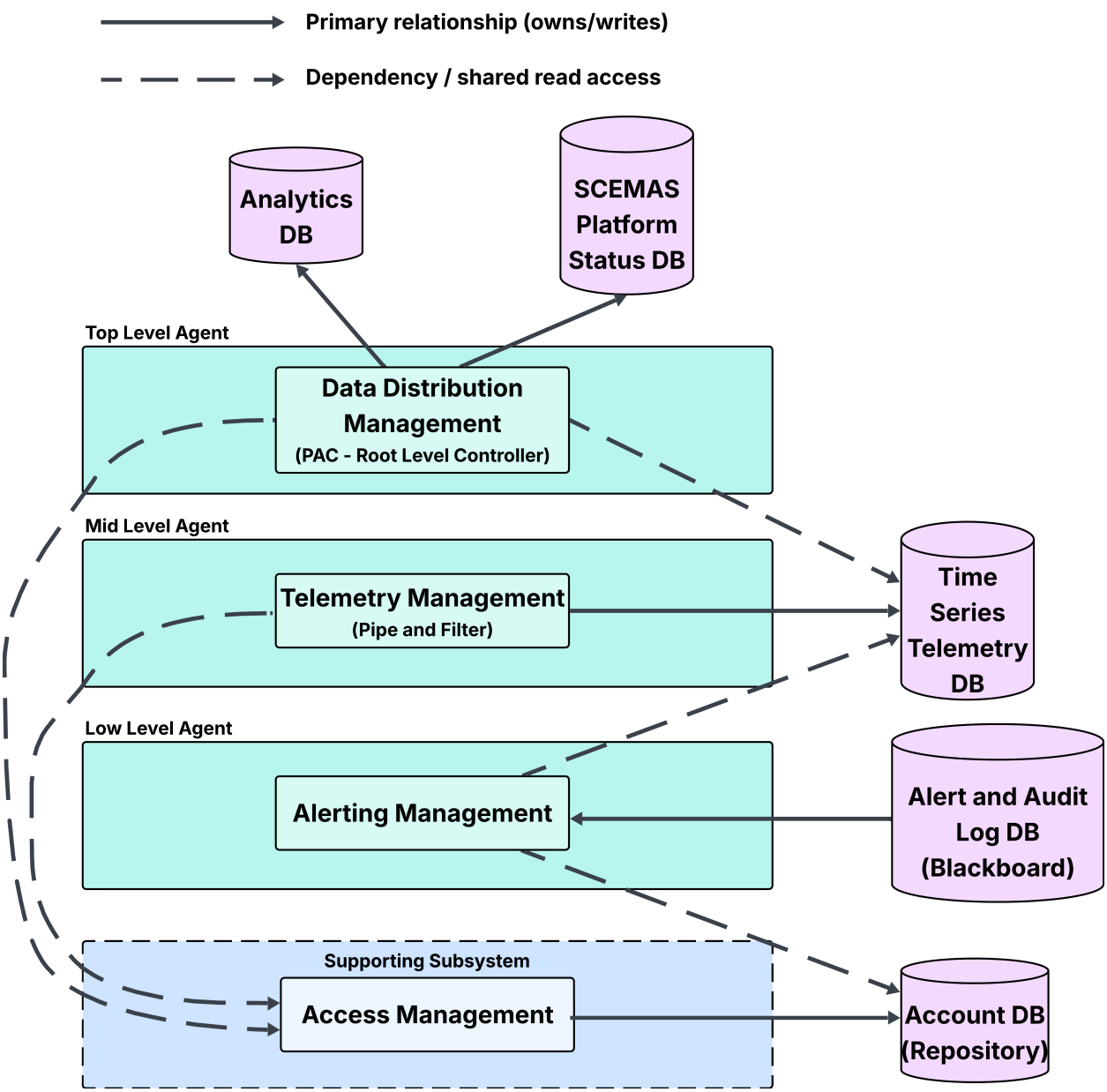


Figure 2: System Architecture

In terms of our sub-system architectural design, we broke our system into three parts; Telemetry Management, Access Management, and Alerting Management. Each has different architectures but corresponds to the overall PAC architecture to communicate.

Telemetry Management although showing Blackboard-like characteristics with an active data store notifying subsequent agents with data changes, the premise of the architecture is not met; independent knowledge sources are not working simultaneously to provide a solution to a non-deterministic problem. Telemetry Management only represents a sequential transformation pipeline to ingest into various sources that handle their specific logic. Raw MQTT message → Schema Validation → Value Range Validation → Timestamp Validation → Persistence → Aggregation. Each stage takes the output of the previous stage as input, transforms and processes it, and passes the resulting structured data packet to the next filter through a pipe, without knowledge of the stages upstream or downstream. This clearly represents the Pipe and Filter architectural style.

Access Management, on the other hand, provides a single authoritative source of truth, with a simple transactional process. User credentials, and RBAC roles are not event-driven or non-deterministic; they are queried deterministically on each authentication request. Authentication requests are clearly initiated by active agents, querying a passive data store that deterministically returns the corresponding role.

Alerting Management clearly represents a blackboard architecture. At its core, it's reactive, data-driven and requires certain knowledge sources to become fulfilled. The lifecycle of an alert, triggered → active → acknowledged → resolved, is a shared data state that multiple consumers react to without being directly coupled to one another. Independent knowledge sources are agents that react to new telemetry, categorize it based on specific thresholds exceeded or anomalous trends, and post it for new rule definitions, and acknowledgement. With alerts being a non-deterministic problem, each agent works to confidently classify the alert event.

Naturally, with a dashboard view, MVC was the first design choice to consider. However, it was shortly eliminated as MVC assumes a single shared model synchronized across all views. With at least three different views; City Operators, Administrators, and Public Users, each serve fundamentally different services with different subsets of data. Meaning a shared model would become a uniform object carrying all user contexts simultaneously, creating high coupling between unrelated concerns.

Batch Sequential was also eliminated as SCEMAS has various latency requirements. For instance, operators must be able to acknowledge alerts within 30 seconds. Batch Sequential processes records in staged batches with no interactive feedback between stages, which is fundamentally incompatible with a live monitoring dashboard that needs real-time alert responses.

Process Control was eliminated easily as it is typically used for the control of an embedded system with continuing physical actions. With this system, none of the observational outputs represented in the dashboard feedback into a measured variable, meaning the closed feedback loop that defines a Process-Control is never formed.

Data-centered architectures, specifically Repository and Blackboard, were considered but eliminated as the overall system architecture. Applying Repository globally would model SCEMAS as a passive data store, providing no inherent mechanism for the interactive, role-differentiated user experiences the system requires. There is no means to express hierarchical agent composition or per-user view differentiation, an operator's personalized alert subscription state must remain independent from another operator's session, which a globally shared repository cannot cleanly represent. Similarly, modelling the entire system as a Blackboard would require all components, including structured account management and role-gated user interfaces to be expressed as knowledge sources reacting opportunistically to a shared data space, which is inappropriate for the deterministic, transactional nature of authentication and RBAC. Both styles are retained at the subsystem level where their properties are directly applicable, but neither is sufficient to describe the system.

3.2 Subsystems

Our main system has been subdivided into three distinct subsystems: Telemetry Management, Alerting Management, and Access Management.

The Telemetry Management subsystem is responsible for all real time collection and ingestion of data, dealing with individual sensor readings, validating its data, monitoring ingestion health and maintaining an up-to-date database of all telemetry related information. This subsystem depends on Access Management to authenticate each incoming IoT device and abides by the pipe and filter architecture style. Telemetry Management and Alerting Management operate as PAC agents under the overall PAC architecture, with Access Management acting as a supporting Repository subsystem.

The Access Management subsystem provides user capabilities such as signing up, logging in, modifying account information, as well as monitoring all security details, including device identity, security permissions,

access status, being the subsystem to maintain the account database, IoT device identity registry and security data. This subsystem does not communicate with any of the others, instead acting as a dependency for both Telemetry Management and Data Distribution Management, which depend on it to verify the identity of IoT devices and authenticated users respectively.

The Alerting Management subsystem defines all functionalities regarding the SCEMAS rule-based alerting system, allowing for definition of rules, managing and distributing of alert notifications, audit logs of all alert events, and the overall management of the alert and audit log database. This subsystem retrieves subscriber information from the shared User Information entity found in the Account Database to determine who to notify upon a threshold violation.

4 Class Responsibility Collaboration (CRC) Cards

Class Name: Telemetry Management Controller	
Responsibility:	**Collaborators:**
Knows Ingest Sensor Streams Knows Validate Telemetry Data Knows Monitor Ingestion Health Knows Time Series Telemetry DB Knows Individual Sensor Reading Knows Access Management	Ingest Sensor Streams Validate Telemetry Data Monitor Ingestion Health Time Series Telemetry DB Individual Sensor Reading Access Management

Class Name: Ingest Sensor Streams (Boundary)	
Responsibility:	**Collaborators:**
Knows Telemetry Management Controller Handles incoming sensor data submission event	Telemetry Management Controller

Class Name: Validate Telemetry Data (Boundary)	
Responsibility:	**Collaborators:**
Knows Telemetry Management Controller Handles telemetry data validation event Handles invalid data rejection event	Telemetry Management Controller

Class Name: Monitor Ingestion Health (Boundary)	
Responsibility:	**Collaborators:**
Knows Telemetry Management Controller Handles ingestion pipeline health monitoring event Displays ingestion health status to system administrators	Telemetry Management Controller

Class Name: Time Series Telemetry DB (Entity)	
Responsibility:	**Collaborators:**
Knows Telemetry Management Controller Knows Individual Sensor Reading Stores timestamped sensor readings Knows Sensor ID Knows Timestamp Knows Zone	Telemetry Management Controller Individual Sensor Reading

Class Name: Data Distribution Management Controller	
Responsibility:	**Collaborators:**
Knows Monitor SCEMAS Platform Status Knows Filter Aggregated Data Knows Provide Public API Knows Visualize City Metrics Knows SCEMAS Platform Status DB Knows Analytics DB Knows Individual Sensor Reading Knows Access Management	Monitor SCEMAS Platform Status Filter Aggregated Data Provide Public API Visualize City Metrics SCEMAS Platform Status DB Analytics DB Individual Sensor Reading Access Management

Class Name: Monitor SCEMAS Platform Status (Boundary)	
Responsibility:	**Collaborators:**
Knows Data Distribution Management Controller Handles platform status monitoring event Displays platform health metrics (uptime, latency, throughput, active sessions) to system administrators Displays security and error status indicators to system administrators	Data Distribution Management Controller

Class Name: Filter Aggregated Data (Boundary)	
Responsibility:	**Collaborators:**
Knows Data Distribution Management Controller Handles aggregated data filter query event Displays filtered data results to city operators	Data Distribution Management Controller

Class Name: Provide Public API (Boundary)	
Responsibility:	**Collaborators:**
Knows Data Distribution Management Controller Handles external API data request event Returns aggregated environmental data to external clients	Data Distribution Management Controller

Class Name: Visualize City Metrics (Boundary)	
Responsibility:	**Collaborators:**
Knows Data Distribution Management Controller Handles city metrics visualization event Displays city metrics on geographical maps and charts to city operators	Data Distribution Management Controller

Class Name: SCEMAS Platform Status DB (Entity)	
Responsibility:	**Collaborators:**
Knows Data Distribution Management Controller Stores platform health and performance records Knows Subsystem Status Knows Uptime Knows Latency Knows Error Rate Knows Timestamp	Data Distribution Management Controller

Class Name: Analytics DB (Entity)	
Responsibility:	**Collaborators:**
Knows Data Distribution Management Controller Stores aggregated city metrics and analytics Knows Zone Knows Metric Type Knows Aggregated Value Knows Timestamp	Data Distribution Management Controller

Class Name: Individual Sensor Reading (Entity)	
Responsibility:	**Collaborators:**
Knows Telemetry Management Controller Knows Alerting Management Controller Knows Data Distribution Management Controller Knows Time Series Telemetry DB Knows Sensor ID Knows Timestamp Knows Metric Type Knows Value Knows Zone	Telemetry Management Controller Alerting Management Controller Data Distribution Management Controller Time Series Telemetry DB

Class Name: Access Management Controller	
Responsibility:	**Collaborators:**
Knows Signup for an Account Knows Login to SCEMAS Platform Knows Modify Account Details Knows Authorize IoT Devices Knows Monitor Access Status Knows Manage Security Permissions Knows Active Session Token Knows Device Identity Registry Knows User Information Knows Account DB	Signup for an Account Login to SCEMAS Platform Modify Account Details Authorize IoT Devices Monitor Access Status Manage Security Permissions Active Session Token Device Identity Registry User Information Account DB

Class Name: Signup for an Account (Boundary)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Handles account signup event Handles invalid signup input event	Access Management Controller

Class Name: Login to SCEMAS Platform (Boundary)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Handles user login authentication event Handles unsuccessful login event	Access Management Controller

Class Name: Modify Account Details (Boundary)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Handles account details modification event Handles invalid modification input event	Access Management Controller

Class Name: Authorize IoT Devices (Boundary)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Handles IoT device authorization event Handles unauthorized device rejection event	Access Management Controller

Class Name: Monitor Access Status (Boundary)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Displays active user sessions and authentication events to system administrators Displays security flags and device authorization status	Access Management Controller

Class Name: Manage Security Permissions (Boundary)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Handles user role assignment event Handles security permission modification event	Access Management Controller

Class Name: Active Session Token (Entity)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Knows Token Value Knows User ID Knows Role Knows Expiry Timestamp	Access Management Controller

Class Name: Device Identity Registry (Entity)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Knows Device ID Knows Device Type Knows Zone Assignment Knows Device Status	Access Management Controller

Class Name: Account DB (Entity)	
Responsibility:	**Collaborators:**
Knows Access Management Controller Knows User Information Stores user account records Knows Email Knows Username Knows Password Knows Role Assignment	Access Management Controller User Information

Class Name: Alerting Management Controller	
Responsibility:	**Collaborators:**
Knows Define Threshold Rules Knows Handle Active Alerts Knows Manage Alert Subscriptions Knows Dispatch Alert Notifications Knows Threshold Rulebook Knows Alert and Audit Log DB Knows Individual Sensor Reading Knows User Information	Define Threshold Rules Handle Active Alerts Manage Alert Subscriptions Dispatch Alert Notifications Threshold Rulebook Alert and Audit Log DB Individual Sensor Reading User Information

Class Name: Define Threshold Rules (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Management Controller Handles threshold rule definition event Handles invalid threshold rule input event	Alerting Management Controller

Class Name: Handle Active Alerts (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Management Controller Displays active alerts with severity and zone to city operators Handles alert acknowledgement and resolution event	Alerting Management Controller

Class Name: Manage Alert Subscriptions (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Management Controller Handles alert subscription configuration event Handles subscription update event	Alerting Management Controller

Class Name: Dispatch Alert Notifications (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Management Controller Handles alert notification dispatch event Handles failed notification delivery event	Alerting Management Controller

Class Name: Threshold Rulebook (Entity)	
Responsibility:	**Collaborators:**
Knows Alerting Management Controller Stores alert threshold rule definitions Knows Metric Type Knows Threshold Value Knows Zone Knows Rule Status	Alerting Management Controller

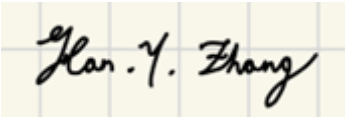
Class Name: Alert and Audit Log DB (Entity)	
Responsibility:	**Collaborators:**
Knows Alerting Management Controller Stores alert event and audit trail records Knows Alert ID Knows Severity Knows Status Knows Timestamp	Alerting Management Controller

Class Name: User Information (Entity)	
Responsibility:	**Collaborators:**
Knows Alerting Management Controller Knows Access Management Controller Knows Account DB Knows User ID Knows Username Knows Name Knows Email Knows Role Knows Account Status	Alerting Management Controller Access Management Controller Account DB

A Division of Labour

Zhang, Han:

- Completed Section 3.2.
- Brainstormed and developed the initial iteration of the high-level system architecture diagram for Section 3.1.
- Updated Use Case Diagram from D1.



Pham, Aaron:

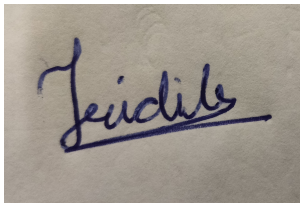
- Section 4: Added and organized all CRC cards following the controller, boundary and entity classes in the Analysis Class Diagram.

Anura, Mithu:

- Section 2: Helped with the creation of the analysis class diagram by identifying several boundary and entity classes.
- All of Section 1, including purpose, system description, and overview.

Khorramijam, Arsia:

- Section 3.1: Determined the overall architecture, provided reasoning and justification, listed design alternatives and tradeoffs, broke system into sub-systems and determined each sub-system's architecture.

Banik, Tridib:

- Section 2: Identified all controller classes and brainstormed boundary and entity classes for Telemetry Management, Alerting Management and Data Distribution Management, and added them in Draw.io covering all requirements from Project Description provided on A2L. I also adjusted positions of classes, directions of arrows (unidirectional or bidirectional), relationship types (association, dependency, or composition), colour-coded different class types (controller, boundary, and entity), adjusted font size and shape size.
- Brainstormed and simplified the high-level system architecture, added colour-coding, adjusted size of shapes, arrows and text for accessible presentation.
- Transferred all content from Word document to Overleaf LaTeX, properly formatted the entire document, including paragraph and table spacing, indexing, justified paragraphs, bolded text, positioning of bullet points or numbers, etc.